

Learn Low-Level Design in 25 Days: Day 10— Observer, Publisher-Subscriber, and Event-Driven Design

The Invisible Magic Behind Real-Time Updates

TechPreneur

Published October 22, 2025

Free: No

Read on Freedium · original

Contents

1. The Core Problem: Tight Coupling and Manual Updates 0
2. The Observer Pattern to the Rescue 0
3. Scaling It Up: Publisher-Subscriber and Event-Driven Design 0
4. Visual Summary 0
5. Try It Yourself 0
6. Summary + What's Next 0

You're scrolling through Trello, and suddenly — a card moves. Someone on your team just dragged a task to "In Progress," and your board updates instantly. No refresh, no reload.

Or think about how Slack delivers new messages across hundreds of open channels in real-time — it's like magic.

That magic is powered by something beautifully simple yet immensely powerful: **the Observer pattern**, often extended into **Publisher-Subscriber** or **Event-Driven** architectures.



Welcome to **Day 10 of Learn Low-Level Design in 25 Days**, where we uncover how modern systems react instead of poll. By the end of this post, you'll understand how to build event-driven systems that keep your app responsive, scalable, and loosely coupled — just like Trello, Slack, or even your favorite trading platform.

1. The Core Problem: Tight Coupling and Manual Updates

Let's start with a common beginner mistake.

You're building a **Weather App**. You have a `WeatherStation` class that fetches data, and a `Display` class that shows it.

You might code something like this:

```
class WeatherStation {
    private float temperature;

    public void setTemperature(float temp) {
        this.temperature = temp;
    }

    public float getTemperature() {
        return temperature;
    }
}

class Display {
    private WeatherStation station;

    public Display(WeatherStation station) {
        this.station = station;
    }

    public void show() {
        System.out.println("Temperature: " + station.getTemperature());
    }
}
```

So far, so good. But what if:

- You add **multiple displays** (desktop, mobile, web)?
- You want to update **all displays** whenever the temperature changes?

You'd end up calling each one manually:

```
display1.show();
display2.show();
display3.show();
```

Suddenly, your `WeatherStation` knows too much about who's watching it. That's tight coupling — brittle, messy, and hard to scale.

2. The Observer Pattern to the Rescue

The **Observer pattern** flips this around.

Instead of the subject (WeatherStation) pushing updates manually, observers subscribe to it and get notified automatically when something changes.

Think of it like **YouTube notifications** — you subscribe to a channel, and whenever a new video drops, YouTube notifies you. The channel doesn't care who you are — it just emits events.

Here's how it looks in design terms:

```
Subject
  + attach(Observer)
  + detach(Observer)
  + notifyObservers()

Observer
  + update()

ConcreteSubject
  - state
  + getState()
  + setState()

ConcreteObserver
  + update()
```

And a simple code version:

```
interface Observer {
    void update(float temperature);
}

class WeatherStation {
    private List<Observer> observers = new ArrayList<>();
    private float temperature;

    public void attach(Observer o) {
        observers.add(o);
    }

    public void detach(Observer o) {
        observers.remove(o);
    }

    public void setTemperature(float temp) {
        this.temperature = temp;
        notifyObservers();
    }

    private void notifyObservers() {
        for (Observer o : observers) {
            o.update(temperature);
        }
    }
}

class PhoneDisplay implements Observer {
    public void update(float temperature) {
        System.out.println("Phone Display: " + temperature);
    }
}

class TVDisplay implements Observer {
    public void update(float temperature) {
        System.out.println("TV Display: " + temperature);
    }
}
```

Now, if the `WeatherStation` changes:

```
WeatherStation station = new WeatherStation();
station.attach(new PhoneDisplay());
station.attach(new TVDisplay());

station.setTemperature(30.5f);
```

Both displays update automatically. No manual calls. No coupling. Pure elegance.

3. Scaling It Up: Publisher-Subscriber and Event-Driven Design

When systems get bigger — say, Slack or Uber — even the Observer pattern starts to stretch. You don't want publishers and subscribers to even know each other.

Enter **Publisher-Subscriber**, often implemented via a **Message Broker** (like Kafka, RabbitMQ, or Redis Streams).

Here's the flow:

```
Publisher ---> Message Queue ---> Subscriber 1
                              ---> Subscriber 2
                              ---> Subscriber 3
```

Each component is **completely decoupled**. The publisher emits an event ("new message sent"), and anyone interested — chat UIs, analytics, notifications — subscribes.

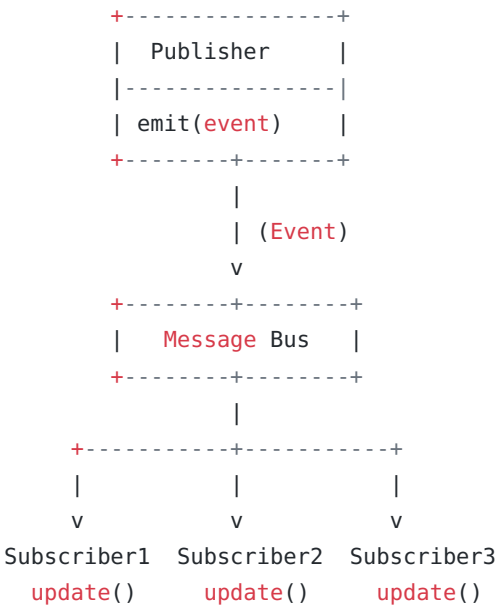
This approach is the backbone of **event-driven architecture**, where systems react to events asynchronously instead of polling or waiting synchronously.

A mini real-world analogy:

- **Trello:** When one user moves a card, the backend emits an event. All connected clients subscribed to that board get an update event and refresh that specific section.
- **Uber:** When a ride is confirmed, multiple subsystems (driver assignment, pricing, notification) all react to that "ride_created" event independently.

This decoupling is what makes these systems **real-time, resilient, and scalable**.

4. Visual Summary



5. Try It Yourself

1. Design Challenge: Imagine a stock trading dashboard that updates live prices for multiple tickers.

- How would you model the system using the Observer pattern?
- What classes would represent the subject and the observers?

2. Real-World Reflection: In your current project, where are updates being "manually pushed"? Could you replace that logic with an event-driven approach?

6. Summary + What's Next

Key Takeaways:

- The **Observer pattern** helps decouple event senders from receivers.
- It's the foundation for **real-time systems** like Trello, Slack, or Uber.
- **Publisher-Subscriber** extends it to distributed systems using event brokers.
- Use it when multiple components need to react to changes independently.
- Avoid it for small, static, or performance-critical components where event overhead isn't worth it.

Tomorrow, we'll level up again — diving into **Case Study: Designing an Elevator System**, and see how to apply all these design patterns in a real-world system that's deceptively complex but incredibly fun to build.

If you're following along, here's the **full series list** — Learn Low-Level Design in 25 Days: A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8) — save it to stay on track!